SubTask 2.1: Functionality Testing

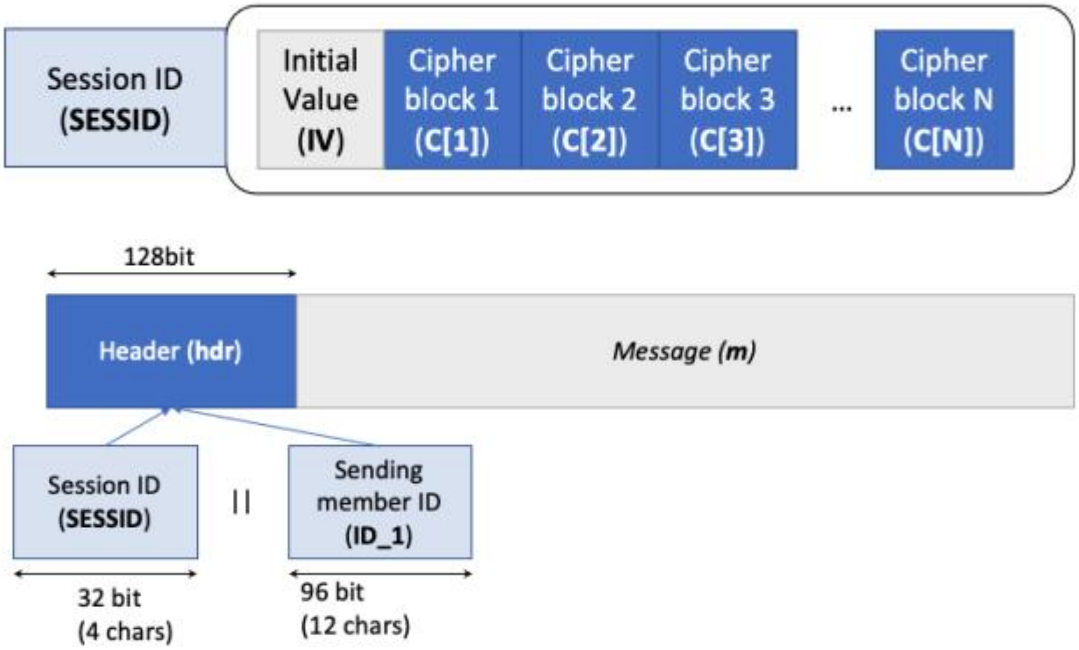
To: kiranjain@gmail.com, misaonakano@gmail.com

Subject: Functionality Testing

Dear Kiran and Misao,

I am writing this email to show you my process of decrypting SESSID||C using the provided group session key (gsk). My method of decrypting involves the usage of the same OpenSSL cryptographic tool as you have.

SESSID||C is a concatenation of the Session ID and the ciphertext as seen in the diagram below.



As the session ID contains 32 bits, or 4 bytes, as 8 bits equals to 1 byte, the Session ID can be extracted using the command below.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/2.1$ head -c 4 cipher1.out > SessionID.txt
```

This command extracts the first 4 bytes of the entire message to obtain the Session ID.

Output:

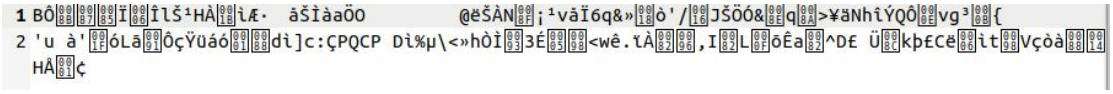
```
1 SES2
```

After obtaining the session ID, the rest of the message can be obtained using the command below.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/2.1$ tail -c+5 cipher1.out > IVCipher.txt
```

This command selects every byte from the end of the file up to the 5th byte. This contains the IV and the cipher text.

Output:



Next, knowing that the IV is the first 128 bits of the new IVCipher text file. The IV can be obtained using the command below.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/2.1$ head -c 16 IVCipher.txt > IV.txt
```

This command extracts the first 16 bytes of the file to obtain the IV.

Output:

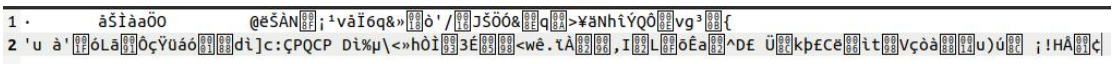


Knowing that the IV take up 16 bytes of IVCipher, the rest of the cipher text is obtained with the command below.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/2.1$ tail -c+17 IVCipher.txt > Cipher.txt
```

The command selects every byte from the end of IVCipher up to the 17th byte, which contains only the message.

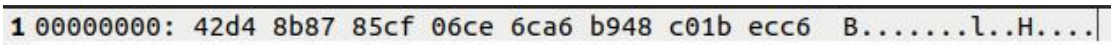
Output:



With the IV and the message obtained, the message can now be decrypted. However, the IV will first need to be converted to a Hexadecimal format as the OpenSSL cryptographic tool takes in HEX values. Therefore, the IV is converted into a Hexadecimal format with the help of the command below.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/2.1$ xxd IV.txt > HexIV.txt
```

Output:



By using the OpenSSL command, the provided gsk and the obtained IV in HEX, the message is decrypted.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/2.1$ openssl enc -d -aes-256-ctr -K AABBCCDDEEFF00112266778899334455001122334455667788 99AABBCCDDEEFF -iv 42d48b8785cf06ce6ca6b948c01becc6 -in Cipher.txt -out Message.txt
```

This command utilizes AES-256-CTR.

Output:

```
1 SES2FIT2093User2My husband loves cats a lot. He has two kitties whose names are Emma and Wendy. They hate eating fish, instead,
  they love dry food.
```

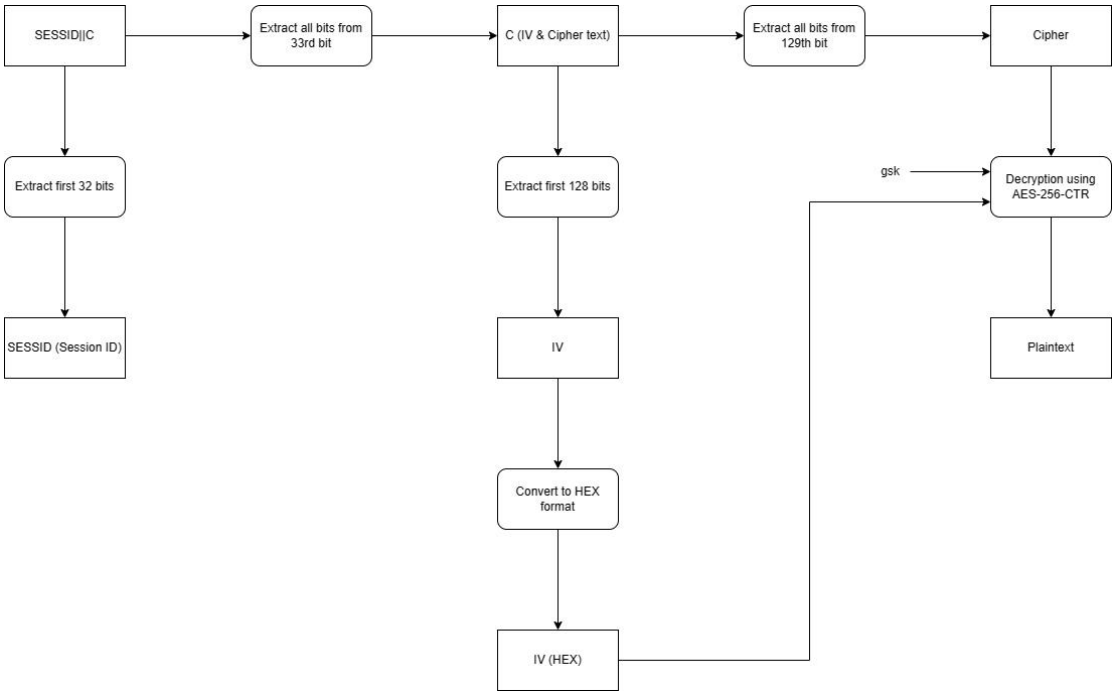
Decrypted Values

Session ID: SES2

Sender ID: FIT2093User2

Message: My husband loves cats a lot. He has two kitties whose names are Emma and Wendy. They hate eating fish, instead, they love dry food.

Block diagram of the decryption method:



I hope you find this email helpful in understanding my decryption process.

Regards,

Braden

SubTask 2.2: Security Evaluation: Confidentiality

To: kiranjain@gmail.com, misaonakano@gmail.com

Subject: Security Evaluation: Confidentiality

Dear Kiran and Misao,

I am writing this email to inform you about the confidentiality security findings from my analysis of the group encryption method.

Security Considerations for CTR Mode of Operation

The block cipher mode of operation that you have used, CTR (Counter Mode), converts a block cipher into a stream cipher. Whenever a counter-initiated value is encrypted, it is then XORed with plaintext, producing the ciphertext block. This mode of operation along with a strong block cipher such as AES-256, provides confidentiality as each plaintext block is encrypted independently. Confidentiality can be achieved by preventing attacks such as Ciphertext-Only Attacks.

Security Vulnerability

Upon further inspection, I found a significant vulnerability in how the IV is generated. The IV is derived from the time of day which might lead to repeating IVs. I will explain how this vulnerability can cause issues.

Using the same method in the prior email, the IV and Ciphertext are obtained with the tail command, followed by the head command to obtain the IV before finally converting the IV to hexadecimal format. These steps are repeated for every message.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/2.2$ tail -c+5 cipher0.out > IVCipher0.txt
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/2.2$ head -c 16 IVCipher0.txt > IV0.txt
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/2.2$ xxd IV0.txt > HexIV0.txt
```

These are the results:

HexIV0.txt | HexIV1.txt | HexIV2.txt | HexIV3.txt | HexIV4.txt

```
1 00000000: 995b 4c9d 0f1f 2efb 213a f024 6511 c1bd  .[L.....!:.$e...
```

HexIV0.txt | HexIV1.txt | HexIV2.txt | HexIV3.txt | HexIV4.txt

```
1 00000000: 995b 4c9d 0f1f 2efb 213a f024 6511 c1bd  .[L.....!:.$e...
```

HexIV0.txt | HexIV1.txt | HexIV2.txt | HexIV3.txt | HexIV4.txt

```
1 00000000: 995b 4c9d 0f1f 2efb 213a f024 6511 c1bd  .[L.....!:.$e...
```

HexIV0.txt | HexIV1.txt | HexIV2.txt | HexIV3.txt | HexIV4.txt

```
1 00000000: 9c62 2c93 0c6c d008 f372 880e 8f4f 43ef  .b,..l...r...OC.
```



As you can see, IVs 0-2 are the exact same. This would cause these messages to produce the same ciphertext block when encrypted, eliminating the randomness of the encryptions while simultaneously enabling attackers to exploit this by analysing the patterns and infer information.

Exploitation by Attacker Zoe

Zoe, an attacker, who eavesdropped on the encrypted group messages, found out about the ID “DeltaZhang00” who sent the third message by overhearing “DeltaZhang00”s private conversation.

By knowing this ID, Zoe could target the messages sent by this user and analysis the patterns of the intercepted ciphertexts. Through this, Zoe could manipulate the messages sent by this specific user or even recover the gsk to decrypt every message, compromising the confidentiality of this conversation.

Recommendations

Here are a few recommendations to fix this vulnerability.

- Use unique IVs: By using a unique IV for each encryption operation, it would be difficult for an attacker to analyse patterns and compromise the conversation. Unique IVs can be generated with a basic random number generator.
- Keep track of IVs: By keeping track of what IVs have been used, we can prevent the occurrence of repeating IVs. This makes it difficult for an attacker to target messages containing the same IVs and obtaining private information.
- Key rotation: Periodically rotating the encryption keys ensures that even if a previous key is compromised, future messages are encrypted with a different key.

I hope you find this email helpful in understanding the potential vulnerabilities of the group encryption method.

Regards,

Braden

SubTask 2.3: Security Evaluation: Integrity/Authenticity

To: kiranjain@gmail.com, misaonakano@gmail.com

Subject: Security Evaluation: Integrity/Authenticity

Dear Kirao and Misao,

I am writing this email to describe the modifications I have made to the group encryption method. I have decided to incorporate a digital signature to the messages to prevent group insider integrity attacks.

Digital Signature

First, using GPG, a key-pair is generated.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/2.3$ gpg --full-generate-key
```

```
gpg (GnuPG) 2.2.19; Copyright (C) 2019 Free Software Foundation, Inc.
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Please select what kind of key you want:
   (1) RSA and RSA (default)
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
  (14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 1024
Requested keysize is 1024 bits
Please specify how long the key should be valid.
         0 = key does not expire
      <n>  = key expires in n days
      <n>w = key expires in n weeks
      <n>m = key expires in n months
      <n>y = key expires in n years
Key is valid for? (0) 0
Key does not expire at all
Is this correct? (y/N) y
```

```
You need a user ID to identify your key; the software constructs the user ID
from the Real Name, Comment and Email Address in this form:
    "Heinrich Heine (Der Dichter) <heinrichh@duesseldorf.de>"

Real name: admin
E-mail address: admin@gmail.com
Comment: My email
You selected this USER-ID:
    "admin (My email) <admin@gmail.com>"
```

Type of Key: RSA and RSA

Key Length: 1024 bits

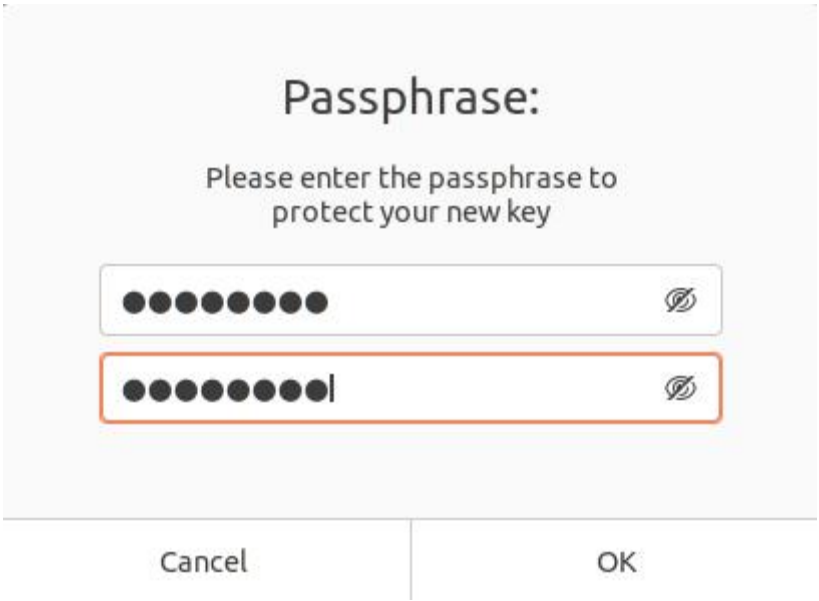
Expiration date: Never

Real Name: admin

Email Address: admin@gmail.com

Comment: My email

After the key is created, the user would enter a password only they know to protect their new key. This ensures that only the user has access to their secret key.




```
public and secret key created and signed.

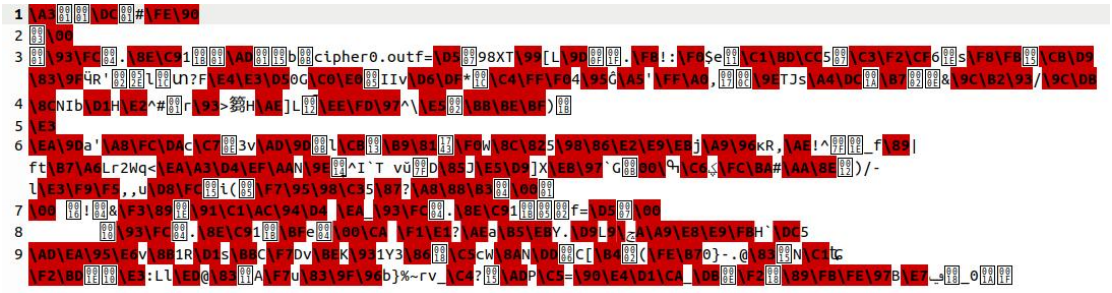
pub   rsa1024 2024-05-10 [SC]
      26F3891E91C1AC94D41DEA5F93FC042E8EC9311B
uid                      admin (My email) <admin@gmail.com>
sub   rsa1024 2024-05-10 [E]
```

The public and secret key are now created.

Next, the ciphertext sent by “DeltaZhang00” is signed using the command below.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/2.3$ gpg --sign cipher0.out
```

Output:



A quick verification is performed to check if the signature is good using the command below.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/2.3$ gpg --verify cipher0.out.gpg
gpg: Signature made Fri 10 May 2024 18:04:23 AEST
gpg:                using RSA key 26F3891E91C1AC94D41DEA5F93FC042E8EC9311B
gpg: checking the trustdb
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: depth: 0  valid:   1  signed:   0  trust: 0-, 0q, 0n, 0m, 0f, 1u
gpg: Good signature from "admin (My email) <admin@gmail.com>" [ultimate]
```

Status: Good signature

The message (cipher0.txt) is then retrieved with the command below.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/2.3$ gpg --output cipher0.txt --decrypt cipher0.out.gpg
File 'cipher0.txt' exists. Overwrite? (y/N) y
gpg: Signature made Fri 10 May 2024 18:04:23 AEST
gpg:                using RSA key 26F3891E91C1AC94D41DEA5F93FC042E8EC9311B
gpg: Good signature from "admin (My email) <admin@gmail.com>" [ultimate]
```

Output:



After that, a detached signature is generated using the command below.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/2.3$ gpg --detach-sign --output sig.bin cipher0.txt
```

The detached signature is also verified using the command below.

```
fit2093@fit2093-vm:~/Desktop/fit2093/Assignment 1 Tasks 2 & 3/2.3$ gpg --verify sig.bin cipher0.txt
gpg: Signature made Fri 10 May 2024 18:16:43 AEST
gpg:                using RSA key 26F3891E91C1AC94D41DEA5F93FC042E8EC9311B
gpg: Good signature from "admin (My email) <admin@gmail.com>" [ultimate]
```

Status: Good signature

As you can observe, both the signature and detached signature produce a “Good signature” outcome when verified if the ciphertext has not been tampered with. I will explain what will happen if an attacker intercepts a message below.

Inside attack

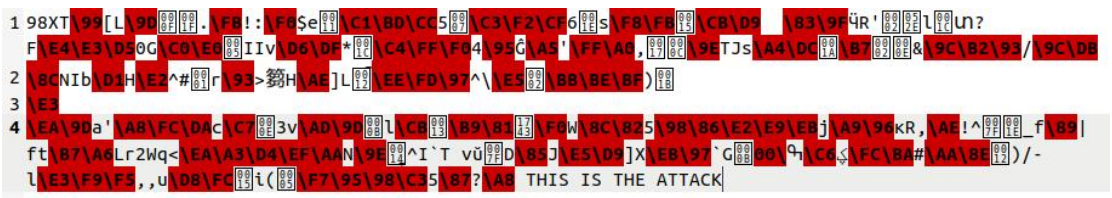
In the case of an attack such as “BobHowes0000” intercepting a message sent by “DeltaZhang00”, this will be the result.

“BobHowes0000” has modified cipher0.txt originally sent by “DeltaZhang00”, as seen below.

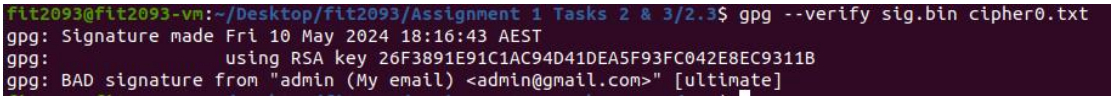
Original by “DeltaZhang00”:



Intercepted by “BobHowes0000”:



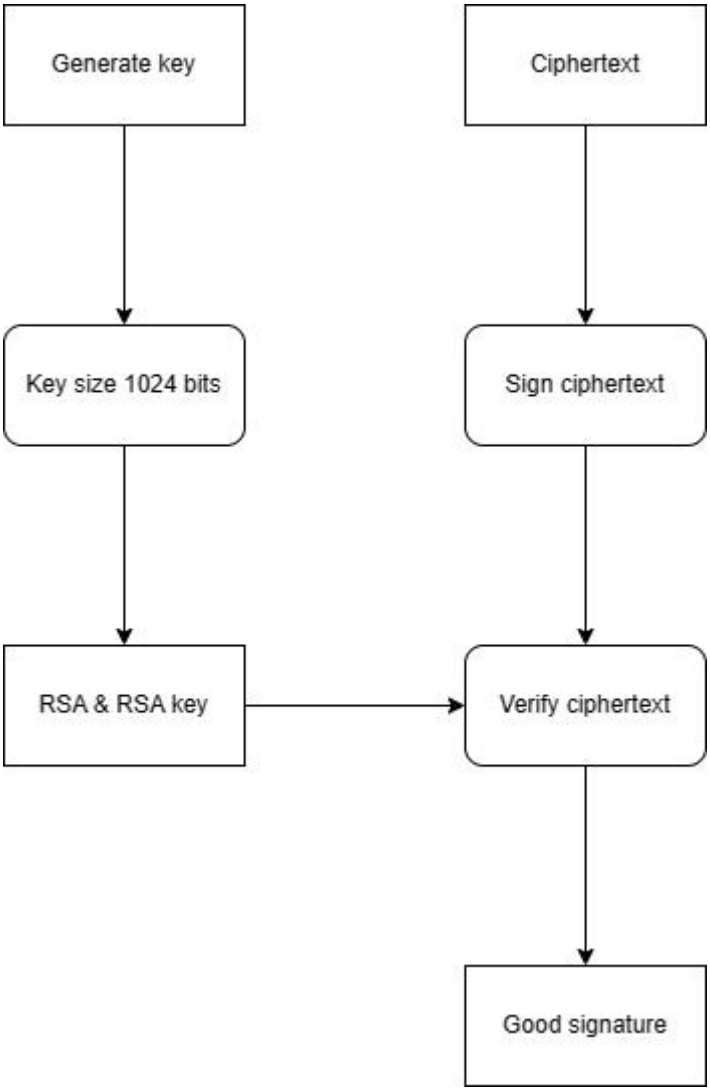
Using the verification command, it can be observed that the message has been compromised and is no longer authentic.



Status: Bad Signature

With this signature, when other members decrypt the message intercepted by “BobHowes0000”, they can recognize that the message has been manipulated and is not the original message sent by “DeltaZhang00”. This maintains the integrity and authenticity of the group encryption method by ensuring that malicious group insiders such as “BobHowes0000” cannot fool other members into thinking that he is honest member “DeltaZhang00”.

Block diagram of the encryption method:



I hope you find this email helpful in understanding how the modified group encryption method works.

Regards,

Braden